AI Privacy Proxy: Architecture, Failure Modes & Engineering Post-Mortem

Project: AI-Privacy-Proxy • Version: 1.0 Audit • Date: October 11, 2026 • Scope: DLP Gateway & Redaction Engine

---

1. Executive Summary & Objective

The primary mission of **AI Privacy Proxy** is to create an on-device Data Loss Prevention (DLP) guardrail that prevents users from inadvertently transmitting sensitive, proprietary, or personal documents (PDFs, code, financials) to cloud-hosted AI platforms (specifically ChatGPT / OpenAI). The system was designed to intercept document uploads transparently, store the files in an on-device local vault, and forward only sanitized text prompts to OpenAI so that ChatGPT can generate responses without ever ingesting confidential files.

Throughout development, initial implementations appeared fully functional in synthetic tests but repeatedly failed in real-world production scenarios. This post-mortem documents the four major failure modes encountered, explains exactly why each assumption broke, and outlines the final technical resolutions implemented in Version 1.

2. Initial Implementation & False Positives

What Was Tried Initially:
The initial architecture assumed a traditional Web 2.0 file upload model: the browser was expected to issue a standard HTTP multipart/form-data POST request containing the file bytes to an upload route (e.g., /upload). The initial proxy addon inspected multipart boundaries, saved the file part, stripped the file field, and forwarded the modified request.

Why It Gave a False Sense of Success:
An automated test suite using a mock local Flask server passed with 100% success. The test server received zero files, the local folder saved the document, and the proxy appeared complete. However, real-world AI applications (like ChatGPT) do not use legacy monolithic file upload models; they use distributed, multi-stage, presigned cloud blob storage architectures.

3. Detailed Breakdown of the 4 Critical Failures

ISSUE: Error 1: 'Unknown Error Occurred' on Real ChatGPT Uploads

Symptom & Premature Assumption: Synthetic tests passed, but attempting to upload an actual PDF to chatgpt.com immediately caused a client-side red banner error: *'Unknown error occurred'*.

Technical Root Cause: Modern ChatGPT web clients do not post files to chatgpt.com/backend-api. Instead, the frontend requests a presigned Azure/AWS blob upload URL and uploads file bytes directly via an HTTP PUT request to *.oaiusercontent.com (e.g. files.oaiusercontent.com). The original proxy addon only listened for multipart POST requests and completely ignored raw binary PUT requests. The file was therefore forwarded directly to cloud blob storage, where incomplete headers or proxy interference caused cloud rejection.

Engineering Resolution: Implemented dedicated interception for HTTP PUT requests directed to *.oaiusercontent.com. The proxy intercepts the raw binary body, hashes and persists the document into storage/, and synthesizes an immediate HTTP 200 OK response with complete CORS headers (Access-Control-Allow-Origin: *, ETag). Exactly zero bytes are transmitted to OpenAI's cloud servers.

ISSUE: Error 2: 'File Size Discrepancy' & Prompt Attachment Indexing Rejection

Symptom & Premature Assumption: Even after returning 200 OK on upload, ChatGPT frontend showed a broken document card, failed to finalize indexing, or threw size mismatch errors.

Technical Root Cause: ChatGPT is a reactive Single-Page App (SPA). After uploading a file, the frontend polls POST /backend-api/files/{file_id} to check server-side retrieval and indexing status. Because the cloud never received the file, OpenAI's backend reported indexing failure. Furthermore, when the user clicked 'Send', the conversation submission payload (/backend-api/.../conversation) included multimodal attachment pointers (file_id, asset_pointer). When OpenAI attempted to resolve those pointers against non-existent cloud files, the conversation failed.

Engineering Resolution: Implemented two-stage protocol simulation: (1) Intercepted /backend-api/files/{file_id} and returned synthetic {'status': 'success', 'ready': true} responses matching OpenAI's schema. (2) Implemented deep recursive JSON sanitization on /backend-api/.../conversation to scrub attachment pointers while preserving prompt text. Verified live: ChatGPT responded *'I don't see a document attached'*, proving complete redaction.

ISSUE: Error 3: Inno Setup Blocked by Windows Security (Error 4551)

Symptom & Premature Assumption: Compiling an installer with Inno Setup failed upon execution with: *'Unable to execute file in the temporary directory. Setup aborted. Error 4551: An Application Control policy has blocked this file.'*

Technical Root Cause: Windows 11 Smart App Control (SAC) and Windows Defender Application Control (WDAC) are active on the host machine. SAC automatically blocks any unsigned executable or DLL extracted into %TEMP%. Inno Setup compilers extract an internal helper (Setup.tmp) to execute, which immediately triggered SAC's zero-trust binary block.

Engineering Resolution: Scrapped compiled .exe installers. Replaced them with native Microsoft-signed script engines (powershell.exe, wscript.exe, certutil.exe) and pure Python execution which run under full platform trust and never drop unsigned binaries into temporary directories.

ISSUE: Error 4: 'Internet Access Denied' & System-Wide Network Lockup

Symptom & Premature Assumption: After installing the background tray service, all browser traffic ceased completely across Google Chrome and Edge with: *'Internet Access Denied / Unable to connect to the proxy server'*.

Technical Root Cause: Four cascading software faults combined to cause total network failure:
1. **Mitmproxy Asyncio Crash:** Mitmproxy 11+ in a worker thread was called as DumpMaster(opts) without passing loop=self.loop, raising an unhandled RuntimeError: no running event loop.
2. **Premature Proxy Activation:** win_proxy.enable_system_proxy() enabled Windows ProxyEnable = 1 before verifying port 8080 was listening. When the proxy crashed, Windows routed all internet traffic to a dead port.
3. **Pythonw Print Crash:** Under pythonw.exe, sys.stdout is None. Any print() statement raised AttributeError: 'NoneType' object has no attribute 'write', crashing windowless processes.
4. **TCP Socket TIME_WAIT Trap:** Single-instance check bound TCP port 48123. On restart, Windows held the port in TCP TIME_WAIT, causing subsequent launches to call sys.exit(0) immediately.

Engineering Resolution: Engineered robust defense-in-depth: (1) Added active TCP socket pre-flight checks: port 8080 is verified accepting connections before Windows proxy is enabled. If port 8080 does not answer within 5 seconds, proxy is automatically aborted. (2) Replaced TCP lock with a native Windows Named Mutex (CreateMutexW), eliminating TIME_WAIT. (3) Redirected stdout/stderr in windowless mode to prevent print crashes. (4) Built immediate emergency reset and uninstall routines.

4. Final Architecture & Verification (Version 1)

To ensure maximum stability and transparency, Version 1 strips away unnecessary background layers and delivers a clean, rock-solid CLI gateway:
• **main.py & run.bat**: Provides an interactive dashboard showing live proxy state, storage folder metrics, and captured documents.
• **proxy/proxy.py**: Intercepts raw PUT cloud uploads, mocks file finalization, and recursively cleans prompt conversations.
• **Failsafe Exit Handler**: Upon typing exit or pressing Ctrl+C, WinINet settings are immediately restored to direct connection (ProxyEnable = 0).
• **Automated Test Suite**: All 15 unit and integration tests across test_chatgpt_flow.py, test_e2e.py, and proxy/test_proxy.py pass with 100% verification.

5. Summary of Lessons Learned

Component	Assumption	Reality & Production Solution
Upload Protocol	Standard multipart POST	Raw binary PUT to cloud presigned endpoints (*.oaiusercontent.com). Must mock 200 OK + CORS.
LLM State	Stripping files from upload is enough	Frontend queries file status and embeds asset pointers into prompt JSON. Must mock status and scrub prompt pointers.
Windows App Control	Inno Setup .exe works everywhere	Windows 11 SAC blocks unsigned temp executables (Error 4551). Use native signed hosts or direct python.
Proxy Safety	Enable proxy immediately on start	Pre-flight socket check is mandatory before registry modification. Failsafe rollback must be guaranteed.